

Trabajo Práctico Integrador

“Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas”

Alumna

Ator, Elizabeth

DNI: 32393628

Tecnicatura Universitaria en Programación

Universidad Tecnológica Nacional.

Programación 1

Junio 2026

Índice

1. Marco Teórico

- 1.1 Listas
- 1.2 Dicionarios
- 1.3 Funciones
- 1.4 Estructuras condicionales y repetitivas
- 1.5 Ordenamientos — Bubble Sort
- 1.6 Estadísticas básicas
- 1.7 Archivos CSV

2. Decisiones Técnicas y

- Arquitectura** 2.1 Estructura del programa
- 2.2 Diagrama de flujo (descripción)
- 2.3 Módulos y funciones

3. Dificultades y Conclusiones

4. Bibliografía / Webgrafía

1. Marco Teórico

1.1 Listas

Una **lista** en Python es una estructura de datos mutable y ordenada que permite almacenar una colección de elementos de cualquier tipo. Los elementos se acceden mediante índices enteros comenzando en 0. Las listas son ideales para representar colecciones donde el orden importa y los datos pueden cambiar con el tiempo.

En este proyecto, la lista `países` almacena todos los registros del dataset. Cada operación (agregar, filtrar, ordenar) trabaja sobre esta lista central.

```
países = [] # Lista vacía
países.append(nuevo_pais) # Agregar al final
len(países) # Cantidad de elementos
```

1.2 Dicionarios

Un **diccionario** es una estructura de datos que asocia claves únicas con valores. En Python se define con llaves `{}` y permite acceso directo a cualquier valor a través de su clave, lo que resulta más legible que usar índices numéricos.

Cada país en el sistema se representa como un diccionario con cuatro claves fijas: nombre, población, superficie y continente. Este diseño hace que el código sea autodocumentado y fácil de mantener.

```
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América"
}
```

1.3 Funciones

Las **funciones** son bloques de código reutilizables que encapsulan una tarea específica. Se definen con la palabra clave `def` y pueden recibir parámetros y retornar valores. El principio de modularización indica que cada función debe tener una única responsabilidad claramente definida.

El programa está organizado en cinco módulos funcionales: manejo del CSV, operaciones CRUD, búsqueda y filtros, ordenamientos y estadísticas. Cada función recibe la lista `países` como parámetro y opera sobre ella.

1.4 Estructuras condicionales y repetitivas

Las **estructuras condicionales** (if / elif / else) permiten tomar decisiones en el flujo del programa según el valor de una expresión booleana. Se usan extensamente para validar entradas del usuario y para despachar opciones del menú principal.

Las **estructuras repetitivas** (while y for) permiten repetir un bloque de código. El bucle while True se usa para mantener el menú activo y para forzar al usuario a ingresar datos válidos. El bucle for se usa para recorrer la lista de países en operaciones de búsqueda, filtrado y estadísticas.

1.5 Ordenamientos — Bubble Sort

El **algoritmo de ordenamiento burbuja** (Bubble Sort) compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Este proceso se repite hasta que la lista queda completamente ordenada. Su complejidad temporal es $O(n^2)$, lo que lo hace simple de implementar y comprender, aunque no es el más eficiente para grandes volúmenes de datos.

En el programa se implementa sobre una **copia** de la lista original para no alterar el orden de los datos en memoria. Soporta ordenamiento ascendente y descendente para los campos nombre (string), población y superficie (enteros).

```
for i in range(n - 1):
    for j in range(n - 1 - i):
        if resultado[j][clave] > resultado[j+1][clave]:
            resultado[j], resultado[j+1] = resultado[j+1], resultado[j]
```

1.6 Estadísticas básicas

Las estadísticas básicas aplicadas en el proyecto incluyen cuatro cálculos implementados con recorridos manuales sobre la lista, sin usar funciones predefinidas de Python como max() o min():

Máximo y mínimo: se inicializa una variable con el primer elemento de la lista y se recorre el resto comparando cada valor. Si se encuentra uno mayor (o menor), se actualiza la referencia. Este algoritmo es $O(n)$ y se aplica tanto para población como para superficie.

Promedio: se acumula la suma de todos los valores en un bucle for y se divide por la cantidad de elementos usando división entera (//) para obtener un resultado entero legible.

Frecuencia por categoría: se usa un diccionario como contador. Por cada país se verifica

si su continente ya está como clave; si está, se incrementa en 1, si no, se inicializa en 1. Este patrón es equivalente al Counter de collections pero implementado manualmente.

1.7 Archivos CSV

Un archivo **CSV** (Comma-Separated Values) almacena datos tabulares en texto plano, donde cada fila representa un registro y cada campo está separado por comas. La primera fila generalmente contiene los encabezados de las columnas.

El programa lee y escribe el CSV manualmente usando `open()`, `readlines()` y `write()`, sin bibliotecas externas, aplicando validaciones por línea para garantizar la integridad del archivo.

```
nombre,poblacion,superficiecontinente
Argentina,45376763,2780400,América
```

2. Decisiones Técnicas y Arquitectura

2.1 Estructura del programa

El programa está contenido en un único archivo **main.py** organizado en cinco módulos lógicos separados por comentarios. Esta estructura facilita la lectura y el mantenimiento del código sin necesidad de múltiples archivos.

1 — CSV	leer_csv, guardar_csv	Lectura y escritura del archivo
2 — CRUD	agregar_pais, actualizar_pais, mostrar_paises	Altas, modificaciones y listado
3 — Filtros	buscar_por_nombre, filtrar_por_*	Búsqueda y filtrado de datos
4 — Orden	ordenar_paises	Bubble Sort asc/desc
5 — Estadísticas	mostrar_estadisticas	Cálculos y conteos
Auxiliar	pedir_entero	Validación de entradas numéricas

2.2 Diagrama de flujo — operaciones principales

El flujo general del programa al ejecutarse es el siguiente:

1. **[INICIO]** — Se ejecuta `main.py`
2. **[CARGA]** — `leer_csv()` lee `paises.csv` y construye la lista de diccionarios

3. **[MENÚ]** — `mostrar_menu()` presenta las 9 opciones disponibles al usuario
4. **[OPCIÓN]** — El usuario ingresa un número del 0 al 9
5. **[DESPACHO]** — Un bloque `if/elif` llama a la función correspondiente
6. **[FUNCIÓN]** — La función ejecuta su lógica, valida entradas y muestra resultados
7. **[GUARDAR]** — Si hubo cambios (agregar/actualizar), `guardar_csv()` persiste los datos
8. **[BUCLE]** — Vuelve al paso MENÚ hasta que el usuario elija la opción 0
9. **[FIN]** — El programa imprime el mensaje de despedida y termina

2.3 Archivos y funciones — detalle

`config.py`

ARCHIVO_CSV — Nombre del archivo CSV usado en todo el sistema.

CONTINENTES_VALID — Lista de los 5 continentes reconocidos por el sistema.

`modulo_csv.py`

`leer_csv()` — Abre el archivo, salta el encabezado, parsea cada línea, valida tipos y campos vacíos. Retorna lista de dicts o `None` si el archivo no existe.

`guardar_csv()` — Sobreescribe el archivo CSV con el encabezado y todos los registros actuales. Retorna `True/False` según éxito.

`modulo_crud.py`

`pedir_entero()` — Auxiliar reutilizable. Solicita un entero al usuario, valida que sea numérico y mayor al mínimo. Soporta valor por defecto para el modo actualizar.

`agregar_pais()` — Valida que no exista un país homónimo, pide los 4 campos con validaciones, crea el dict y llama a `guardar_csv()`.

`actualizar_pais()` — Busca el país por nombre, muestra valores actuales, permite actualizar población y/o superficie (Enter conserva el valor).

`mostrar_paises()` — Imprime una tabla formateada con columnas alineadas y separadores de miles.

modulo_filtros.py

buscar_por_nombre() — Filtro por substring case-insensitive sobre el campo nombre. Muestra todos los resultados parciales.

filtrar_por_rango() — Función interna que abstrae la lógica de rango numérico. Recibe el nombre del campo como parámetro para evitar duplicación.

filtrar_por_continente() — Valida el continente ingresado y muestra los países correspondientes.

filtrar_por_poblacion() — Llama a `filtrar_por_rango()` con el campo `poblacion`.

filtrar_por_superficie() — Llama a `filtrar_por_rango()` con el campo `superficie`.

modulo_orden_estadisticas.py

ordenar_paises() — Implementa Bubble Sort sobre una copia de la lista. Permite elegir criterio (nombre, población, superficie) y dirección (asc/desc).

mostrar_estadisticas() — Calcula máx/mín de población y superficie con recorridos manuales. Calcula promedios con suma manual. Genera conteo por continente con diccionario acumulador y barra visual.

main.py

mostrar_menu() — Imprime el menú de 9 opciones en consola.

main() — Carga el CSV, ejecuta el bucle del menú y despacha cada opción a la función del módulo correspondiente mediante `imports`.

3. Dificultades y Conclusiones

Dificultades encontradas

Una de las primeras dificultades fue el manejo correcto de la codificación del archivo CSV. Los nombres de países con caracteres especiales (como Japón o África) generaban errores al leer el archivo si no se especificaba `encoding='utf-8'` en la función `open()`. La solución fue incorporar este parámetro de forma sistemática en todas las operaciones de lectura y escritura.

Otro desafío fue el diseño de las validaciones de entrada. Inicialmente cada función tenía su propio código repetido para pedir y validar números enteros. Esto se resolvió creando la función

auxiliar *pedir_entero()* en *modulo_crud.py*, que centraliza esa lógica y acepta parámetros configurables para adaptarse a distintos contextos (mínimo permitido, valor por defecto para actualizaciones).

El Bubble Sort presentó una dificultad conceptual al querer soportar tanto orden ascendente como descendente. La solución fue usar una variable booleana *invertir* que modifica la condición de intercambio, evitando duplicar el algoritmo.

Organizar el proyecto en múltiples archivos también requirió planificar con cuidado las dependencias entre módulos para evitar importaciones circulares. La solución fue definir las constantes en *config.py* como punto de referencia común, y establecer un flujo claro donde *main.py* importa de todos los módulos pero los módulos no se importan entre sí excepto cuando es estrictamente necesario.

Conclusiones

El desarrollo de este trabajo integrador permitió consolidar los conceptos centrales de Programación 1 aplicándolos en un sistema real y funcional. La principal conclusión técnica es que la combinación de **listas de diccionarios** resulta una estructura natural y expresiva para representar datos tabulares: la lista provee el orden y el acceso por índice, mientras el diccionario provee el acceso semántico por nombre de campo, eliminando la necesidad de recordar posiciones numéricas.

La **modularización en archivos separados** demostró ser fundamental para la organización y el mantenimiento del código. Distribuir las funciones en *config.py*, *modulo_csv.py*, *modulo_crud.py*, *modulo_filtros.py* y *modulo_orden_estadisticas.py* permitió que cada parte del sistema sea independiente y fácil de ubicar. La función auxiliar *pedir_entero()* es el ejemplo más claro del principio una función, una responsabilidad: al centralizar la validación de entradas numéricas, cualquier corrección se aplica automáticamente a todas las funciones que la usan.

Implementar el **Bubble Sort** manualmente, sin *sorted()*, fue valioso para comprender que un algoritmo de ordenamiento es esencialmente un problema de comparación e intercambio repetido. La variable *invertir* para soportar ambas direcciones sin duplicar el algoritmo fue una decisión de diseño que reflejó el valor de pensar antes de codificar.

Finalmente, el manejo de **archivos CSV** reforzó la importancia de no asumir que los datos externos son siempre correctos. Las validaciones línea por línea en *leer_csv()* garantizan que un archivo malformado no rompa el programa, sino que informe el problema y continúe procesando el resto de los datos.

4. Bibliografía / Webgrafía

- **Documentación oficial de Python 3**

<https://docs.python.org/3/>

- **Python — Estructuras de datos (listas y diccionarios)**

<https://docs.python.org/3/tutorial/datastructures.html>

- **Python — Lectura y escritura de archivos**

<https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>

- **Python — Funciones definidas por el usuario**

<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

- **GeeksForGeeks — Bubble Sort Algorithm**

<https://www.geeksforgeeks.org/bubble-sort/>

- **Wikipedia — Formato CSV**

https://es.wikipedia.org/wiki/Valores_separados_por_comas

- **Real Python — Reading and Writing CSV Files**

<https://realpython.com/python-csv/>

Links del proyecto

- **Repositorio GitHub:**

https://github.com/Eliza-ator/tpi_programacion1

- **Video explicativo/ demostrativo:**

<https://youtu.be/sUT7G9mll98>